StreetCare AI — technical architecture

The technology in each tier, the exact library versions in the project environment, the modules that use them, and how the tiers talk to each other

Versions from the project venv (myenv) · settings from config/config.py

1 Client tier

Web browser
HTML · JS · WebSocket

renders Streamlit pages;
no logic, no local storage

Streamlit — 1.67
run_dashboard.py · dashboard/ (6 page modules)

st.navigation + st.Page — only the pages in the role's list
st.session_state keeps the user and token on the server side
st.form · st.spinner · st.dataframe · st.badge · .streamlit/config.toml theme

dashboard/api_client.py — httpx 0.28
the single HTTP client

`call_service(method, path, query, payload) → JSON`
adds X-Session-Token · timeout 300 s · spinner for long calls
401 → clears the session, back to sign-in · ≥ 400 → st.error(detail)

REST · JSON · X-Session-Token · :8081

2 API tier

Uvicorn — 0.54
ASGI server

`0.0.0.0:8081`
`run_api_service.py`
checks 13 required files
before importing routes

FastAPI — 0.162
api/routes.py · service_application

public_router
`/auth/signin · /auth/signout`
guarded_router
`9 routes · Depends(authorize_request)`
report_locks: one threading.Lock per open report
handlers → 502: ValueError · genai APIError · httpx.HTTPError

Pydantic — 2.14
api/schemas.py

4 request models: SignIn, Review,
Decision, Chat · 8 response models
Literal[...] enums built from config lists
StringConstraints: strip, min / max length
→ 422 before any handler runs

access/ — stdlib
passwords · sessions · permissions

`hashlib.pbkdf2_hmac('sha256', 200 000)`
16-byte salt · hmac.compare_digest
`secrets.token_urlsafe(24)`
`PERMISSION_BY_ROUTE → 403`
`authorize_district → 403`

invoke / resume · *ask*

3 Orchestration & AI

LangGraph — 1.2
agentic/graph.py · nodes.py · state.py

`StateGraph(ReportReviewState)`
`TypedDict · llm_call_count: Annotated[int, operator.add]`
10 nodes · 7 add_edge chains · 2 add_conditional_edges
`route_report_decision → ORDER / INSPECT / REFER`
`interrupt(review_id) → invoke(Command(resume=decision))`
nodes return only the keys they write

SqliteSaver — ckpt-sqlite 3.1
langgraph-checkpoint-sqlite

`sqlite3.connect(checkpoint.db,`
`check_same_thread=False)`
a checkpoint after every node
resumable after a restart
or from another process

google-genai — 3.0
agentic/language_model.py · agents.py

`genai.Client · key from .env`
temperature 0.2 · max 8 192 tokens
automatic function calling off
`parse_labelled_response`
→ fixed labelled fields or error
each reply → llm_calls (tokens, ms)

chat/
Street Care assistant

`conversation_memory`
the asker's own last turns
`street_care_assistant`
classify · retrieve · answer
`record_answer`
report facts by role, no LLM

facts for each node

4 Domain, ML & knowledge

tools/ — plain Python
deterministic — never calls the LLM

`report_data_tools.py`
`screening_tools.py`
`evidence_tools.py`
`prediction_tools.py`
`retrieval_tools.py`
`decision_tools.py`
`review_document_tools.py`

scikit-learn — 1.9
modeling/ · data_pipeline/feature_builder.py

`Pipeline(ColumnTransformer(`
`numbers: SimpleImputer(median) → StandardScaler`
`categories: SimpleImputer(most_frequent) → OneHotEncoder))`
LogisticRegression (max_iter 3000) → repeat likelihood
RandomForestRegressor (300 trees, leaf 10, features 0.7) → crew hours
TfidfVectorizer → LogisticRegression → 9 sentence types
joblib 1.6 (compress 3) · pandas 3.1 · numpy 2.5 · seed 1313

ChromaDB + sentence-transformers — 1.5 · 6.2
knowledge/

pypdf 7.0 → page text → regex section headings
passages of 700 chars with 120 carried; a requirement is never split
PersistentClient · collection street_care_documents · cosine
all-MiniLM-L6-v2 embeddings on CPU (torch 2.15) · 384 dimensions
exact section read-back for citations; casebook excluded from chat
`standard_reading`
requirement figures parsed from each standard as Decimal

5 Persistence

SQLAlchemy — 2.1
persistence/ — the only module that queries

declarative_base · 6 ORM models (models.py)
create_engine(sqlite:///database/streetcare.db, check_same_thread=False)
repository.py: 13 functions · select + join / outerjoin

streetcare.db
SQLite · 6 tables
users · runs · reviews
decisions · calls · chat

checkpoint.db
SQLite · WAL
checkpoints · writes
paused review state

chroma.sqlite3
ChromaDB
205 passages + vectors
16 PDFs

*.pkl · *.json
output/models/
3 fitted pipelines
2 feature schemas

processed/*.csv
pandas
5 clean extracts
+ quality report

6 Build, test & run

Python — 3.14 · 3.11
local venv · Docker image

decimal (ROUND_HALF_UP)
threading · secrets · hmac
python-dotenv 1.2

pytest — 9.2
tests.py

50 tests: data, ML, RAG, agents,
API and dashboard
stand-in model replaces Gemini

Entry points
run from Project/

`run_data_pipeline · run_model_training · run_knowledge_indexing`
`run_database_setup · run_api_service · run_dashboard`
each checks the files it needs first

Docker
python:3.11-slim

one image, two processes
EXPOSE 8080
entrypoint script

GitHub Actions
.github/workflows/build.yml

SonarCloud quality scan
platform trigger on push

Interfaces & protocols

Browser ↔ Streamlit
HTTP + WebSocket · :8080

Streamlit → FastAPI
REST · JSON · :8081

header X-Session-Token

FastAPI → LangGraph
in-process invoke / resume

LangGraph → Gemini
HTTPS · google-genai SDK

chat → Gemini
HTTPS · 1 call per question

tools → ChromaDB
in-process · SQLite file

tools → models
joblib.load at import

all → database
SQLAlchemy ORM sessions

graph → checkpoints
shared sqlite3 connection

first embedding
Hugging Face Hub download

Cross-cutting concerns

Configuration
config/config.py: one source
for every value; .env for the key

Security
PBKDF2 hashes · random tokens
route → permission → district

Errors
HTTPException 401/403/404/409/422
one handler → 502, run FAILED

Concurrency
per-report lock; threads share
SQLite with check_same_thread off

Audit
llm_calls: tokens + latency
chat_turns: every reply

Determinism
temperature 0.2 · labelled fields
seed 1313 · Decimal half-up

Cost bound
≤ 7 LLM calls per review
≤ 1 per chat question

Client · Service code · Security · LLM / orchestration · Machine learning · Storage · Tooling · call / data flow · LLM call